Testes Unitários com JUnit

Herysson R. Figueiredo
herysson.figueiredo@ufn.edu.br

Objetivos da aula

- Distinguir técnica de teste (caixa-preta/caixa-branca) de nível ou estratégia de teste.
- Compreender o objetivo, o escopo e o isolamento esperado em testes unitários.
- Relacionar critérios funcionais e estruturais à seleção de casos de teste de unidade.
- Implementar testes automatizados com JUnit Jupiter, usando assertivas e testes parametrizados.
- Interpretar falhas e cobertura de código sem confundir cobertura com ausência de defeitos.

Fonte: adaptado de MARTINS (2016, cap. 3 e 5); PRESSMAN; MAXIM (2021, cap. 19); JUnit User Guide (2026).

Onde estamos?

1 Introdução Teste de Software › 2 Caixa-branca › 3 Caixa-preta › **4 Teste unitário** › 5 Integração

Agora vamos transformar critérios de projeto de teste em testes executáveis.

Técnica ≠ nível de teste

01 TÉCNICAS — Como escolher os dados?

Caixa-preta / funcional
- parte da especificação
- equivalência, limites, decisão, estados...

Caixa-branca / estrutural
- parte do código
- nós, ramos, condições, caminhos...

02 NÍVEIS — Em que escala testar?

Unidade → Integração → Validação → Sistema

O nível indica o escopo do software considerado. A técnica indica como os casos serão projetados.

Um teste unitário pode ser projetado com critérios funcionais, estruturais ou ambos.

Fonte: MARTINS (2016, cap. 2 e 3); PRESSMAN; MAXIM (2021, cap. 19).

Do código ao sistema



Começamos pela menor unidade e ampliamos progressivamente o escopo.

Fonte: MARTINS (2016, cap. 3), adaptado de Pressman.

O que é teste unitário?

Teste dedicado à menor unidade de software, buscando defeitos de lógica e programação em um escopo pequeno e controlado.

A ESCOPO

Uma unidade por vez.

B OBJETIVO

Verificar comportamento esperado e condições internas relevantes.

C RESULTADO

Falhas localizadas, mais fáceis de reproduzir e depurar.

Fonte: MARTINS (2016, cap. 3 e 5); PRESSMAN; MAXIM (2021, cap. 19).

O que pode ser uma unidade?

Função

menor operação funcional

Procedimento

rotina isolável

Método

comportamento de uma classe

Classe

conjunto coeso de operações

Componente

quando suficientemente pequeno e isolável

A “unidade” depende da arquitetura e do paradigma adotado.

Fonte: MARTINS (2016, cap. 3 e 5).

Por que testar unidades pequenas?

01 Gerenciamento

Casos, entradas e resultados ficam mais fáceis de organizar quando o foco é pequeno.

02 Depuração

Quando o teste falha, o espaço de busca pelo defeito é menor.

03 Paralelização

Unidades diferentes podem ser testadas por pessoas/processos diferentes.

Escopo menor não elimina defeitos — mas torna a investigação mais controlável.

Fonte: MARTINS (2016, cap. 3), com base em MYERS (2004).

Isolamento é parte da ideia

Ao testar uma unidade, dependências externas não deveriam dominar o resultado do teste.

Banco de dados

evitar acesso real

Arquivos

evitar I/O externo

Rede / APIs

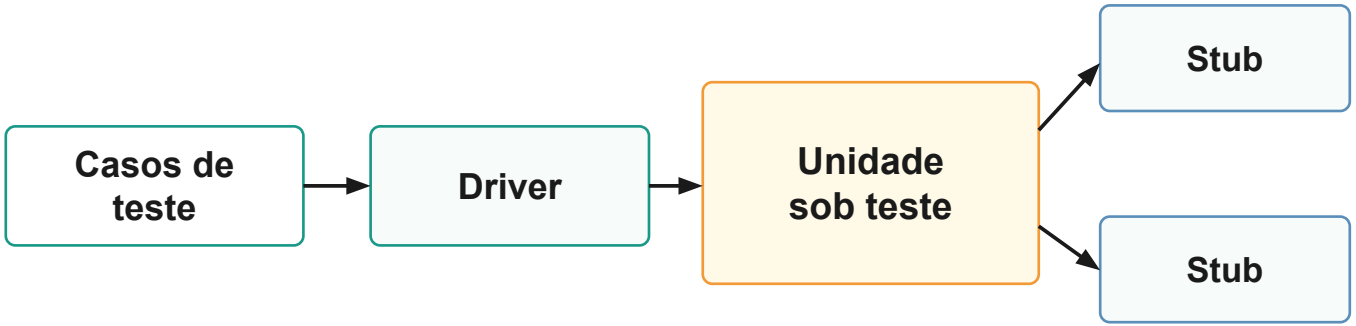
evitar indisponibilidade

Relógio / ambiente

controlar variáveis externas

Fonte: MARTINS (2016, cap. 5).

Ambiente de teste de unidade



Um Stub é um objeto ou função falsa que substitui uma dependência real no código e retorna respostas pré-definidas e fixas para que a unidade que está sendo testada possa rodar isoladamente

- Interface
- Dados locais
- Condições-limite
- Caminhos independentes
- Tratamento de erros

Fonte: adaptado de PRESSMAN; MAXIM (2021, cap. 19) e MARTINS (2016, cap. 3).

Driver e stub: por que aparecem no teste unitário?

D DRIVER

Substitui a unidade que chamaria a unidade sob teste.

Em testes automatizados, a própria classe de teste frequentemente cumpre esse papel: prepara entradas, chama o método e observa resultados.

S STUB

Substitui uma dependência chamada pela unidade sob teste.

Devolve respostas simples e controladas, suficientes para que o comportamento da unidade possa ser testado isoladamente.

A ideia central é controlar as dependências para atribuir a falha à unidade testada.

Fonte: MARTINS (2016, cap. 3), com base em Pressman.

O que devemos exercitar em uma unidade?

1	**Interface**	Entradas e saídas da unidade.
2	**Estruturas de dados**	Valores locais e manipulação interna.
3	**Condições-limite**	Fronteiras e casos extremos.
4	**Caminhos independentes**	Lógica de controle relevante.
5	**Tratamento de erros**	Exceções e respostas anormais.

Fonte: PRESSMAN; MAXIM (2021, cap. 19); MARTINS (2016, cap. 3).

Teste unitário pode usar caixa-preta e caixa-branca

Da especificação — funcional

- Partições de equivalência
- Valores-limite
- Tabelas de decisão
- Estados / regras de negócio

Pergunta: quais comportamentos precisam ser demonstrados?

Do código — estrutural

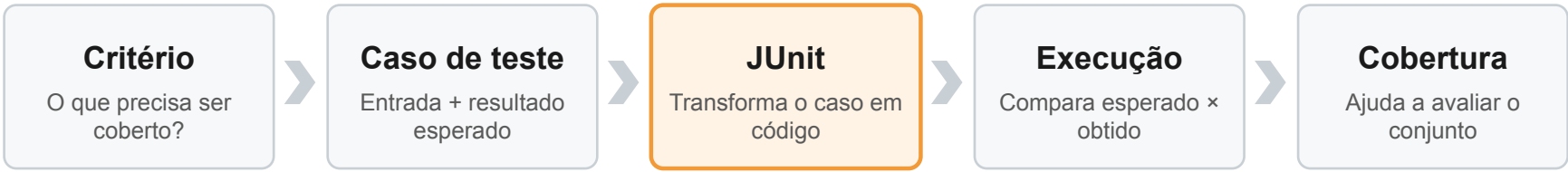
- Comandos e nós
- Ramos e condições
- Caminhos independentes
- Tratamento de erros

Pergunta: quais partes da lógica precisam ser exercitadas?

Uma suíte robusta combina comportamento esperado + estrutura relevante.

Fonte: MARTINS (2016, cap. 3); PRESSMAN; MAXIM (2021, cap. 19).

Critério / teste automatizado



Automatizar não substitui o projeto dos casos de teste — automatiza sua execução.

JUnit: onde entra?

JUnit é um framework para escrever e executar testes automatizados na JVM.

1 Escrever

Anotações e API para descrever testes.

2 Verificar

Assertivas comparam esperado e obtido.

3 Executar

IDE, Maven e outras ferramentas executam a suíte.

Fonte: MARTINS (2016, cap. 5); JUnit User Guide 6.1.3 (2026).

JUnit 6 — visão rápida

JUnit Platform

Base para descobrir e executar testes na JVM. Integra frameworks, IDEs e ferramentas de build.

JUnit Jupiter

Modelo de programação usado para escrever testes modernos com @Test, assertivas, extensões e testes parametrizados.

JUnit Vintage

Compatibilidade temporária com testes JUnit 3/4. O mecanismo Vintage está depreciado.

Nesta aula: JUnit Jupiter.

Fonte: JUnit User Guide 6.1.3 (2026).

Pré-requisitos da prática

01 Java

JUnit 6 requer Java 17 ou superior em tempo de execução.

02 Projeto

Usaremos um projeto Java com Maven para gerenciar a dependência.

03 IDE

IntelliJ, Eclipse, NetBeans ou VS Code podem executar testes da JUnit Platform.

A lógica dos testes permanece a mesma mesmo quando o ambiente de desenvolvimento muda.

Fonte: JUnit User Guide 6.1.3 (2026).

Dependência Maven

```
<dependency>
        <groupId>org.junit.jupiter</groupId>
        <artifactId>junit-jupiter</artifactId>
        <version>6.1.3</version>
        <scope>test</scope>
</dependency>
```

Escopo test: a biblioteca é necessária para compilar/executar testes, não para a aplicação em produção.

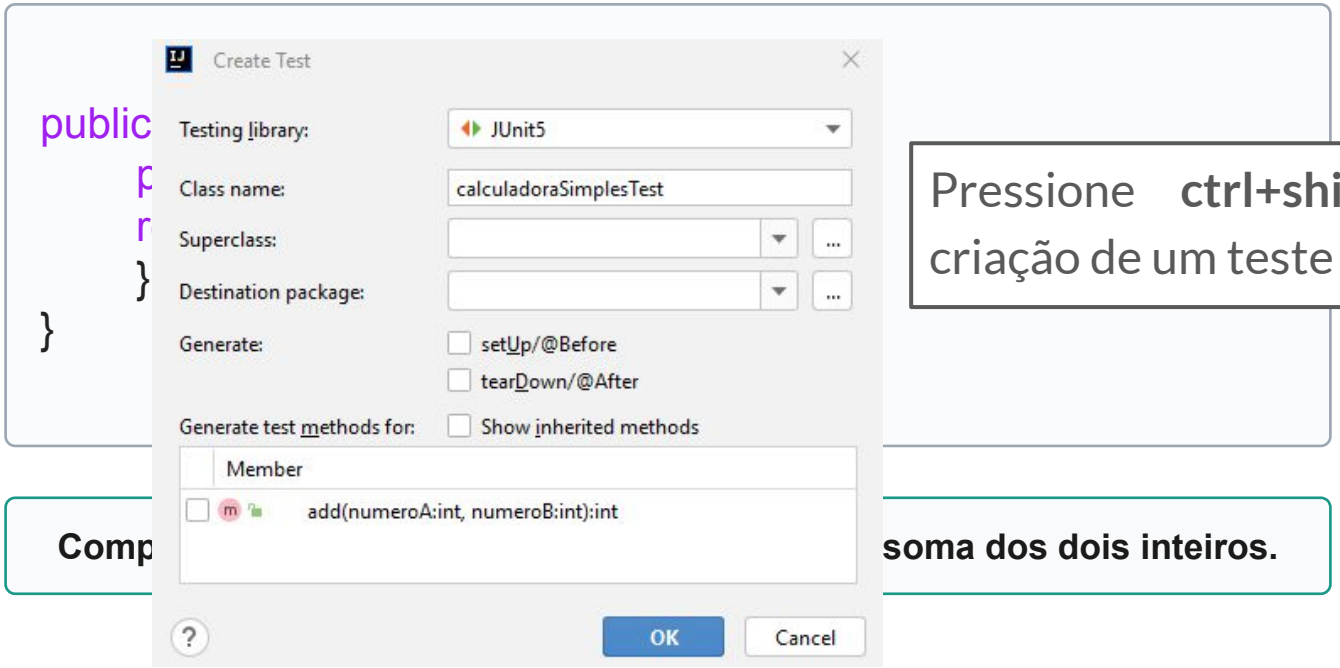
Fonte: JUnit User Guide 6.1.3 — Build Support (2026).

Primeiro exemplo: CalculadoraSimples

```
public class CalculadoraSimples {
        public int somar(int a, int b) {
        return a + b;
        }
}
```

Comportamento esperado: somar(a,b) devolve a soma dos dois inteiros.

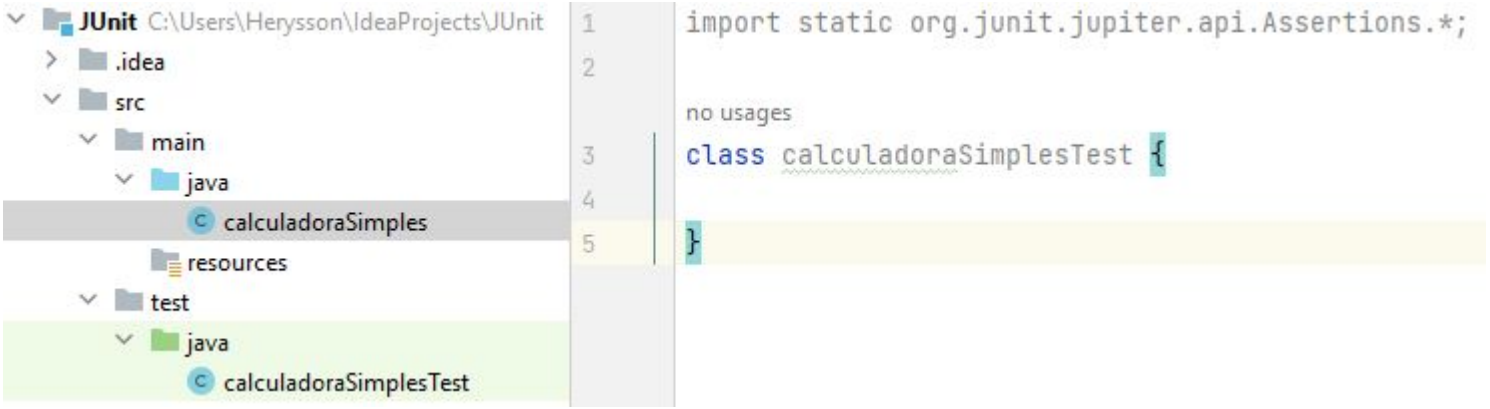
Primeiro exemplo: CalculadoraSimples



Pressione **ctrl+shift+t** para criação de um teste

Comp **soma dos dois inteiros.**

Primeiro teste



Fonte: estrutura do exemplo alinhada ao JUnit User Guide (2026).

Primeiro teste



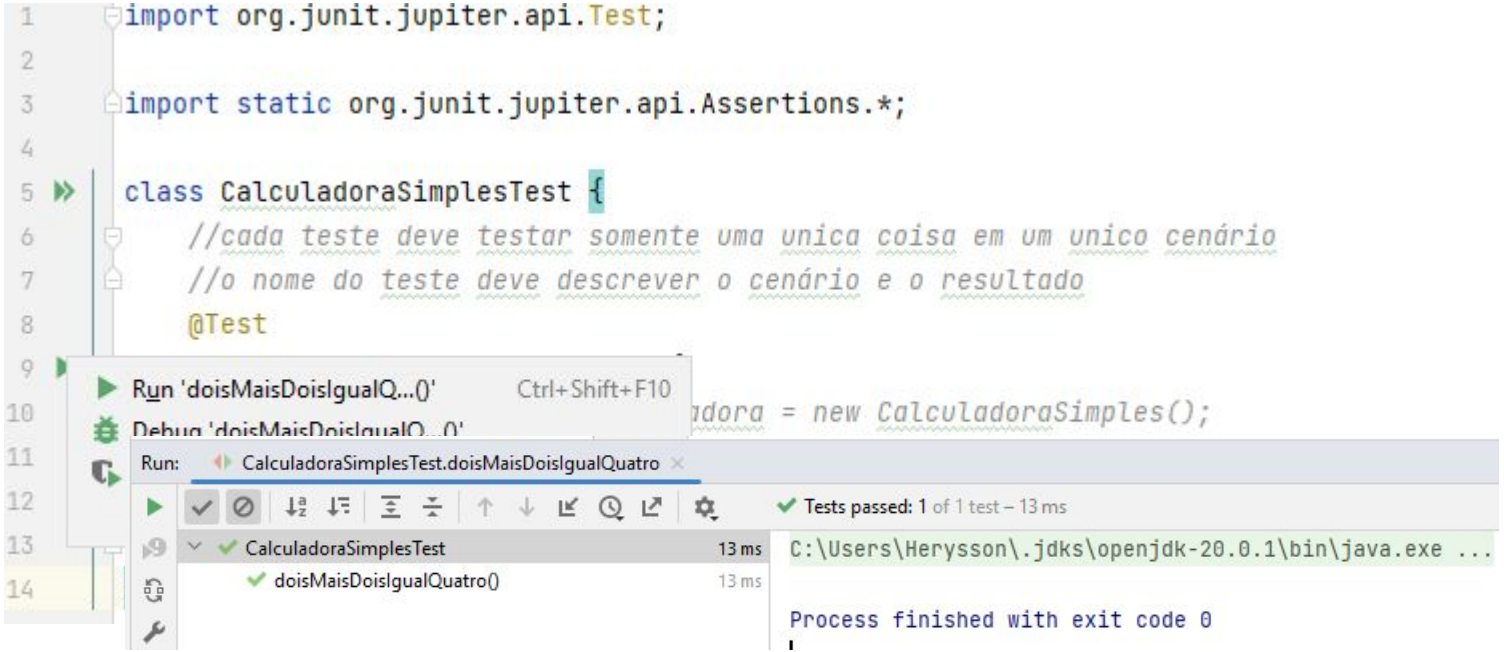
```java
import org.junit.jupiter.api.Test;

import static org.junit.jupiter.api.Assertions.*;

class CalculadoraSimplesTest {
    //cada teste deve testar somente uma unica coisa em um unico cenário
    //o nome do teste deve descrever o cenário e o resultado
    @Test
    void doisMaisDoisIgualQuatro(){
        //CalculadoraSimples calculadora = new CalculadoraSimples();
        var calculadora = new CalculadoraSimples();
        assertEquals( expected: 4,calculadora.add( numeroA: 2, numeroB: 2));
    }
}
```

Fonte: estrutura do exemplo alinhada ao JUnit User Guide (2026).

Primeiro teste



Fonte: estrutura do exemplo alinhada ao JUnit User Guide (2026).

Primeiro teste

```
1   public class CalculadoraSimples {
2
        1 usage
3       public int add (int numeroA, int numeroB){
4           return numeroA-numeroB;
5       }
6   }
```

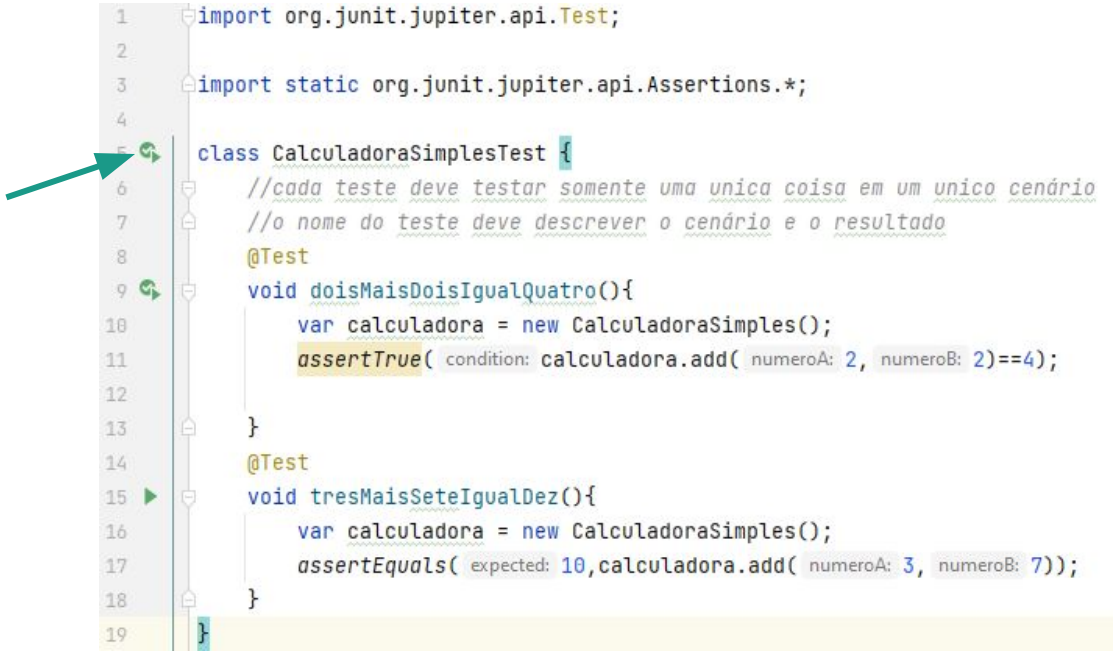
Alterando a classe CalculadoraSimples

```
8       @Test
9
10                                  adora = new CalculadoraSimples();
11                                  adoraSimples();
12                                  uladora.add( numeroA: 2, numeroB: 2));
13
14  }
```

Run 'doisMaisDoisIgualQ...()' Ctrl+Shift+F10
Debug 'doisMaisDoisIgualQ...()'
Run 'doisMaisDoisIgualQ...()' with Coverage
Modify Run Configuration...

Fonte: estrutura do exemplo alinhada ao JUnit User Guide (2026).

Primeiro teste



```java
import org.junit.jupiter.api.Test;

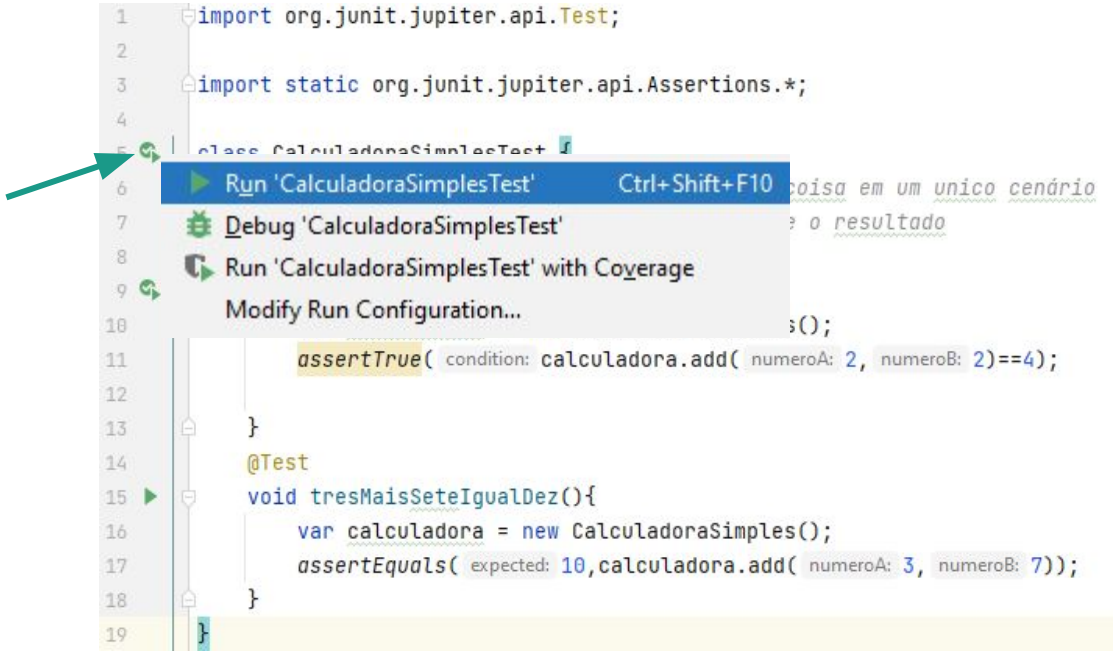
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraSimplesTest {
    //cada teste deve testar somente uma unica coisa em um unico cenário
    //o nome do teste deve descrever o cenário e o resultado
    @Test
    void doisMaisDoisIgualQuatro(){
        var calculadora = new CalculadoraSimples();
        assertTrue( condition: calculadora.add( numeroA: 2, numeroB: 2)==4);

    }
    @Test
    void tresMaisSeteIgualDez(){
        var calculadora = new CalculadoraSimples();
        assertEquals( expected: 10,calculadora.add( numeroA: 3, numeroB: 7));
    }
}
```

Fonte: estrutura do exemplo alinhada ao JUnit User Guide (2026).

Primeiro teste



Fonte: estrutura do exemplo alinhada ao JUnit User Guide (2026).

Anatomia de um teste unitário

```
@Test
void deveSomarDoisInteiros() {
        var calc = new CalculadoraSimples();   // preparar
        int resultado = calc.somar(2, 2);      // executar
        assertEquals(4, resultado);            // verificar
}
```

@Test

Marca o método como teste.

Nome

Descreve comportamento/cenário.

Assertiva

Decide se o teste passa ou falha.

Fonte: JUnit User Guide (2026); elaboração própria.

Arrange – Act – Assert

ARRANGE

Preparar objeto, entradas e estado inicial.

ACT

Executar exatamente o comportamento que está sendo testado.

ASSERT

Comparar o resultado obtido com o esperado.

A estrutura ajuda a separar preparação, ação e verificação — tornando o teste legível.

Fonte: elaboração própria — padrão de organização amplamente usado; exemplo didático.

Assertivas: o oráculo do teste

assertEquals

esperado == obtido

assertNotEquals

valores diferentes

assertTrue / assertFalse

condição booleana

assertNull / assertNotNull

presença/ausência de valor

assertThrows

exceção esperada

assertAll

grupo de verificações

A assertiva transforma o resultado observado em PASSA ou FALHA.

Fonte: JUnit User Guide 6.1.3 — Assertions (2026).

Quando o teste falha?

```
public int somar(int a, int b) {

return a - b;   // defeito inserido
}
```

```
assertEquals(4, calc.somar(2, 2));
```

Esperado: 4

Obtido: 0

O teste não “corrige” o defeito: ele evidencia que o comportamento observado não corresponde ao esperado.

Fonte: MARTINS (2016, cap. 5); JUnit User Guide (2026).

Um bom nome descreve o comportamento

Pouco informativo

test1()
calculo()
testaSoma()

O leitor precisa abrir o teste para entender o cenário.

Mais informativo

deveSomarDoisInteiros()
deveRejeitarNotaNegativa()
deveAplicarFreteGratisAcimaDe200()

O nome já comunica o comportamento esperado.

Nome do teste = documentação executável do comportamento.

Fonte: elaboração própria — recomendação didática de legibilidade.

Um teste deve ser pequeno e focado

Teste um comportamento observável por cenário — sem misturar várias histórias independentes no mesmo método.

Evite

Um único teste que soma, divide, valida nota, acessa arquivo e ainda verifica vários comportamentos sem relação.

Prefira

Testes curtos, independentes e com causa de falha fácil de localizar.

Um cenário pode ter mais de uma assertiva quando todas verificam o mesmo comportamento.

Fonte: princípio de isolamento coerente com MARTINS (2016, cap. 5).

@BeforeEach: preparação repetida

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
class CalculadoraSimplesTest {
        private CalculadoraSimples calc;
        @BeforeEach
        void preparar() {
                calc = new CalculadoraSimples();
        }
        @Test
        void deveSomarDoisInteiros() {
                 assertEquals(4, calc.somar(2, 2));
        }
}
```

Fonte: JUnit User Guide 6.1.3 — Annotations (2026).

Exemplo guiado: classificação de notas

Intervalo	Resultado
nota < 0	exceção
0 ≤ nota < 60	F
60 ≤ nota < 70	D
70 ≤ nota < 80	C
80 ≤ nota < 90	B
90 ≤ nota ≤ 100	A
nota > 100	exceção

Antes de escrever @Test: quais valores representam melhor cada comportamento?

Implementação da classe Nota

```
public class Nota {
        public char determinarConceito(double nota) {
        if (nota < 0 || nota > 100)
                throw new IllegalArgumentException("Valor inválido");
        else if (nota < 60) return 'F';
        else if (nota < 70) return 'D';
        else if (nota < 80) return 'C';
        else if (nota < 90) return 'B';
        else return 'A';
        }
}
```

Fonte: elaboração própria — exemplo didático.

Testando exceções com assertThrows

```
@Test
void menosUmRetonaExcecao(){
   var nota = new Nota();
   assertThrows(IllegalArgumentException.class,
           () ->{
              nota.determinaLetra(-1);
           });
}
```

Uma exceção esperada também é comportamento — e deve ser testada explicitamente.

Fonte: JUnit User Guide 6.1.3 — Exception Handling (2026).

Antes de programar testes: quais casos escolher?

A

1. Especificação

Quais classes de comportamento existem?

Quais fronteiras mudam o resultado?

B

2. Estrutura

Quais ramos e condições existem?

Há tratamento de erros?

C

3. Conjunto

Quais testes cobrem comportamento + estrutura com o mínimo de redundância?

Fonte: MARTINS (2016, cap. 2); ligação com as aulas de caixa-preta e caixa-branca.

Particionamento de equivalência aplicado à classe Nota

Classe	Intervalo	Exemplo
Inválida 1	nota < 0	-1
F	0 a < 60	30
D	60 a < 70	65
C	70 a < 80	75
B	80 a < 90	85
A	90 a 100	95
Inválida 2	nota > 100	101

Fonte: MARTINS (2016, cap. 2); PRESSMAN; MAXIM (2021, cap. 19).

Análise de valor limite aplicada à classe Nota

Fronteira	Logo abaixo	No limite	Logo acima
0	-0,1	0	0,1
60	59,9	60	60,1
70	69,9	70	70,1
80	79,9	80	80,1
90	89,9	90	90,1
100	99,9	100	100,1

Os valores exatos dependem do tipo de dado e da precisão relevante no domínio.

Fonte: MARTINS (2016, cap. 2); PRESSMAN; MAXIM (2021, cap. 19).

Matriz de casos para a classe Nota

CT	Entrada	Esperado	Objetivo
1	-0,1	exceção	inválido inferior
2	0	F	fronteira inferior
3	59,9	F	antes de 60
4	60	D	fronteira 60
5	70	C	fronteira 70
6	80	B	fronteira 80
7	90	A	fronteira 90
8	100	A	fronteira superior
9	100,1	exceção	inválido superior

Fonte: elaboração própria — casos derivados das partições e fronteiras do exemplo.

Implementando casos representativos

```
class NotaTest {
        private final Nota nota = new Nota();
        @Test void zeroRetornaF() {
                assertEquals('F', nota.determinarLetra(0));
        }
        @Test void sessentaRetornaD() {
                assertEquals('D', nota.determinarLetra(60));
        }
        @Test void noventaRetornaA() {
                assertEquals('A', nota.determinarLetra(90));
        }
}
```

Fonte: JUnit User Guide (2026); casos derivados do exemplo anterior.

Testes parametrizados: quando vários casos têm a mesma lógica

@ParameterizedTest executa o mesmo método de teste várias vezes com argumentos diferentes.

Quando ajuda

- muitos valores-limite
- tabelas de entrada → saída
- mesma verificação repetida

Benefício didático

Separa claramente:
DADOS DO TESTE
+
LÓGICA DA VERIFICAÇÃO

Fonte: JUnit User Guide 6.1.3 — Parameterized Tests (2026).

Exemplo com @CsvSource

```
class CalculadoraSimplesTest {
    private CalculadoraSimples calc;
    @BeforeEach
    void preparar() {
        calc = new CalculadoraSimples();
    }
    @ParameterizedTest
    @CsvSource({
            "2, 2, 4",
            "0, 5, 5",
            "-2, 3, 1",
            "10, -4, 6",
            "-3, -7, -10"
    })
    void deveSomarDoisInteiros(int numeroA, int numeroB, int esperado) {
        assertEquals(esperado, calc.add(numeroA, numeroB));
    }
}
```

Fonte: JUnit User Guide 6.1.3 — Parameterized Tests (2026).

Entradas inválidas também podem ser parametrizadas

```
@ParameterizedTest
@ValueSource(doubles = {-1, 101})
void deveRejeitarNotasForaDoIntervalo(double entrada) {
        assertThrows(IllegalArgumentException.class,
                () -> nota.determinarLetra(entrada));
}
```

O caso inválido não “deve passar sem erro”: o comportamento correto é lançar a exceção esperada.

Fonte: JUnit User Guide 6.1.3 — Parameterized Tests e Exception Handling (2026).

Executar e interpretar o resultado

✓ PASSA

Resultado esperado = resultado obtido.

A execução daquele caso não revelou divergência.

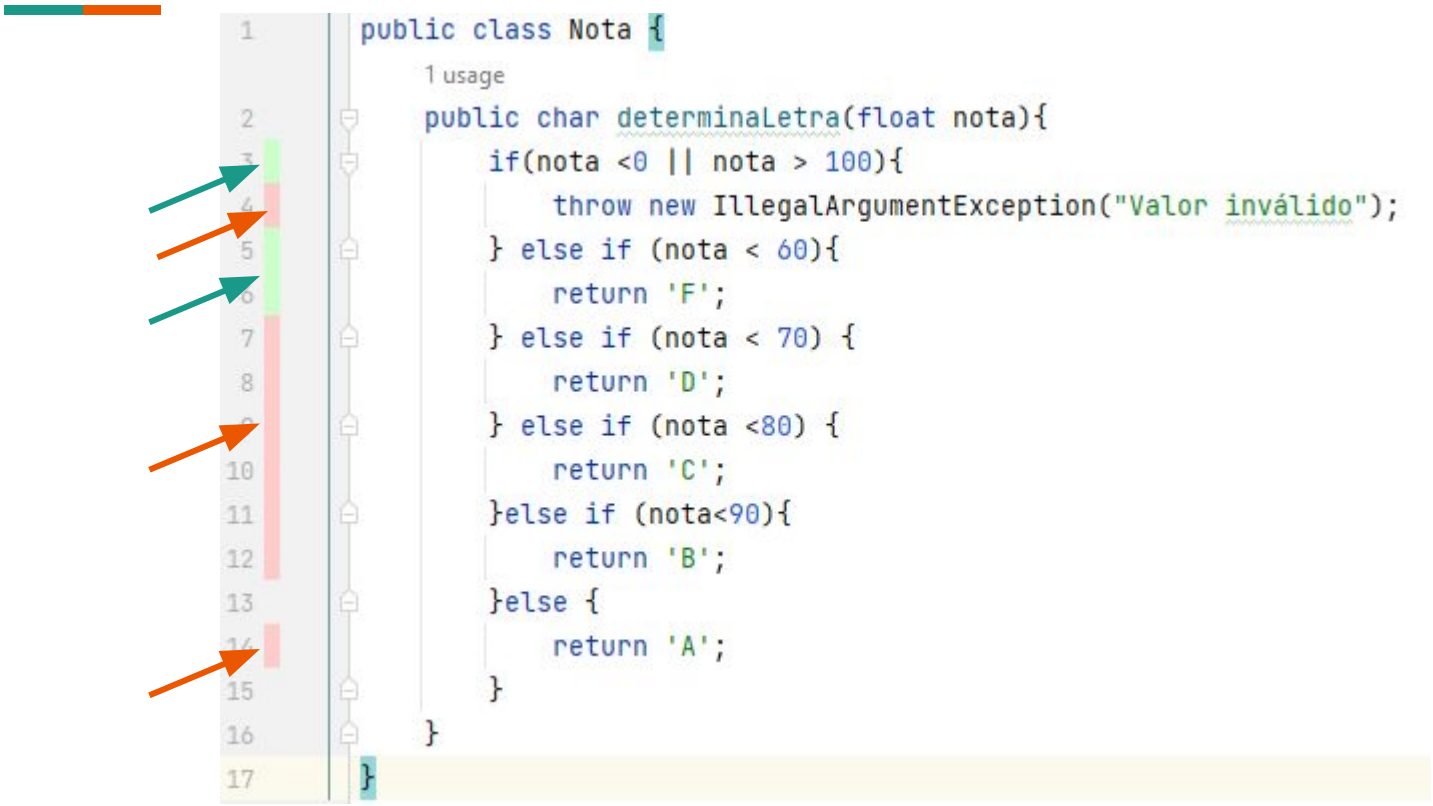
! FALHA

Assertiva falhou ou ocorreu uma exceção inesperada.

Há algo a investigar no código, no teste ou na especificação.

Teste falhando é informação: reproduza, isole e investigue a causa.

Fonte: MARTINS (2016, cap. 5); JUnit User Guide (2026).


1 usage
public class Nota {
public char determinaLetra(float nota){
if(nota <0 || nota > 100){
throw new IllegalArgumentException("Valor inválido");
} else if (nota < 60){
return 'F';
} else if (nota < 70) {
return 'D';
} else if (nota <80) {
return 'C';
}else if (nota<90){
return 'B';
}else {
return 'A';
}
}
}

Cobertura de código: o que estamos medindo?

L Cobertura de linhas/comandos

Quantas instruções executáveis foram exercitadas pela suíte?

R Cobertura de ramos

Quantas saídas possíveis das decisões foram percorridas?

Cobertura ajuda a localizar partes não exercitadas; ela não mede diretamente a qualidade das assertivas.

Fonte: PRESSMAN; MAXIM (2021, cap. 19); MARTINS (2016, cap. 2); elaboração própria.

100% de cobertura não significa “sem defeitos”

Cobertura responde: “o código foi exercitado?”
Não responde sozinha: “o comportamento foi verificado corretamente?”

Pode haver

assertiva errada, requisito incompleto, dados insuficientes, defeitos não ativados.

Portanto

use cobertura como evidência complementar ao projeto criterioso dos casos.

Cobertura alta + casos ruins ainda pode produzir uma suíte fraca.

Fonte: MARTINS (2016, cap. 2); PRESSMAN; MAXIM (2021, cap. 19).

Atividade prática I calcularFrete

```
public double calcularFrete(double valor, boolean premium) {
        double frete = 20.0;
        if (valor >= 200) {
                frete = 0.0;
        }
        else if (premium) {
                frete = 10.0;
        }
        return frete;
}
```

- Derive casos por partições e valor-limite.
- Implemente os testes em JUnit.
- Cubra todos os ramos do método.
- Explique quais casos são necessários para comportamento e quais para cobertura.

Fonte: elaboração própria — atividade de integração com as aulas anteriores.

Desafio — melhore a suíte

Comece com três testes que cubram os ramos. Depois refine a suíte pensando nos limites.

Questão	Pista
Qual valor está na fronteira?	200
O que testar em torno dela?	199,99 / 200 / 200,01
premium importa quando valor ≥ 200?	não para o resultado deste método
Dá para parametrizar?	sim — separe dados da lógica da asserção

Fonte: elaboração própria — continuação da atividade prática.

Atividade prática II e III

Atividade no https://github.com/Herysson/Qualidade-e-Teste-de-Software/blob/main/Atividades/Testes_Unitario.md

1. identifique as classes de equivalência e os valores-limite;
2. defina os casos de teste;
3. implemente os testes utilizando JUnit;
4. utilize testes parametrizados quando houver vários casos semelhantes;
5. utilize `assertThrows()` para comportamentos inválidos;
6. execute os testes com cobertura e analise se o conjunto criado é suficiente.

Fonte: elaboração própria — atividade de integração com as aulas anteriores.

Resumo

- Teste unitário é um nível/estratégia de teste; caixa-preta e caixa-branca são técnicas de projeto.
- A unidade deve ser testada em escopo pequeno e, sempre que possível, isolado de dependências externas.
- Os casos devem nascer de critérios: equivalência, limites, ramos, condições, caminhos e tratamento de erros.
- JUnit automatiza a execução e a verificação dos casos por meio de testes, assertivas e recursos como parametrização.
- Cobertura ajuda a avaliar o conjunto, mas não substitui bons casos de teste nem boas assertivas.

Fonte: síntese baseada em MARTINS (2016), PRESSMAN; MAXIM (2021) e JUnit User Guide (2026).

Próximo passo: teste de integração

Unidades corretas isoladamente não garantem que funcionarão corretamente quando combinadas.



A próxima pergunta é: as interfaces e interações entre essas unidades funcionam como esperado?

Fonte: MARTINS (2016, cap. 3); PRESSMAN; MAXIM (2021, cap. 19).

Referências

PRESSMAN, Roger S.; MAXIM, Bruce R. Engenharia de software: uma abordagem profissional. 9. ed. Porto Alegre: AMGH, 2021.

MARTINS, Marcos Danilo Chiodi. Testes de software. 1. ed. Rio de Janeiro: SESES, 2016.

JUNIT TEAM. JUnit User Guide: version 6.1.3. 2026. Disponível em: https://docs.junit.org/6.1.3/. Acesso em: 26 out. 2026.

JUNIT TEAM. Build Support; Writing Tests; Annotations; Exception Handling; Parameterized Tests. JUnit User Guide 6.1.3, 2026.